UPSC Notes Portal — Day-by-Day Build Plan

Subject-wise notes & question papers · Admin upload · Student view/download · Cloudflare + Supabase + R2
Assumption: **~2–3 hours per day** (21 working days). Double daily hours to finish in ~10–11 days.

> **Goal:** Replace WhatsApp/Telegram PDF hunting with one link — subject tiles (Polity, History, …), day/topic filters, in-browser PDF view, download. Only you upload; 1000+ students read via CDN (handles 10,000+ visitors). PDF storage: **10 GB free** on R2, then ~₹1.25/GB/month (50 GB cap in ops).

Week 1 — Accounts, backend, storage

Day	Focus	Tasks (checklist)	Done when
Day 1	Accounts & repo	• Create GitHub account/repo (private) for code only • Sign up: Cloudflare, Supabase (free tier) • Install Node.js LTS, Git, VS Code/Cursor • Clone/init project folder; add .gitignore (.env, node_modules)	Empty repo pushed; three cloud accounts ready
Day 2	Supabase database	• Create Supabase project (region closest to India if available) • Tables: subjects, materials (day, topic, title, type, file key, size, published) • Seed subject rows: Polity, History, Geography, Economy, etc. • Enable Row Level Security: public read if published; no public write	Subjects visible in Supabase table editor; RLS on
Day 3	Admin auth	• Supabase Auth: email + strong password (your admin only) • RLS policies: INSERT/UPDATE/DELETE only for your user UUID • Document URL + anon key in local .env (never commit secrets)	You can log in via Supabase Auth test; policies block anon writes
Day 4	Cloudflare R2	• Create R2 bucket (e.g. upsc-materials) • Enable public access via custom domain or r2.dev subdomain for PDF URLs • Create API token (R2 read/write); store in Supabase secrets for later • Set CORS: allow your site origin + PUT for uploads	Test file uploadable in R2 dashboard; URL opens in browser

Day	Focus	Tasks (checklist)	Done when
Day 5	Upload API (Edge Function)	• Function init_upload: verify JWT → return presigned PUT URL + material id • Function complete_upload: verify file exists → set published = true • Test with curl or Postman using admin token	One PDF uploads end-to-end without using the website UI yet
Day 6	Frontend scaffold	• Create Astro + Tailwind app under apps/web • Env: Supabase URL, anon key, public R2 base URL • Layout: mobile-first header, footer, institute name	npm run dev shows home page locally
Day 7	Subject tiles (student)	• Home page: grid of subject tiles from Supabase • Each tile links to /subject/[slug] • Basic styling: large tap targets for phones	All subjects clickable; list page loads (can be empty)

Week 2 — Student UI, admin, go live

Day	Focus	Tasks	Done when
Day 8	Material list	• Subject page: fetch published materials for subject • Cards: title, Day N, topic, type (Notes / Paper) • Sort by day_number, then newest	Uploaded test PDFs appear on subject page
Day 9	PDF view & download	• Page /view/[id] with PDF.js embedded viewer • Download button → direct R2 URL (attachment disposition) • Fallback: “Open in new tab” for older phones	Student can read without downloading; download works
Day 10	Filters	• Filter by day (dropdown), content type • Topic search (client-side or Supabase ilike)	Find “Day 12 Polity” in under 10 seconds
Day 11	Admin login UI	• Route /admin/login (not linked from student nav) • Supabase client signInWithPassword • Protect /admin/* routes if no session	Only you reach admin dashboard after login
Day 12	Admin upload UI	• Form: subject, day, topic, title, type, file • Call init_upload → PUT file to R2 → complete_upload • Progress indicator for large PDFs	Upload from browser without Postman

Day	Focus	Tasks	Done when
Day 13	Admin manage content	• Table of all materials: edit metadata, unpublish, delete • Storage meter: sum file_size_bytes → show used / 50 GB; warn at 45 GB	Wrong upload fixed without database access
Day 14	Deploy MVP	• Connect GitHub → Cloudflare Pages; build command for Astro • Set production env vars in Pages • Smoke test on real phone (mobile data) • Share *.pages.dev with 10–20 pilot students	Public URL works; pilot feedback collected

Week 3 — Scale, content ops, launch

Day	Focus	Tasks	Done when
Day 15	Performance	• Cache-Control on catalog API or generate static catalog.json on publish • Lazy-load PDF.js only on view page • Cloudflare Web Analytics snippet (optional)	Pages feel fast on mobile
Day 16	PDF compression	• Document Ghostscript workflow OR compress before upload • Target 30–40% smaller scans to save GB and cost	Process written; one batch compressed and re-uploaded
Day 17	Batch upload (optional)	• CSV import: filename, subject, day, topic, type • Multi-file select on admin upload	10 PDFs uploaded in one session
Day 18	Security pass	• Confirm no service-role key in frontend bundle • R2: no bucket listing; UUID object keys • Enable Supabase 2FA on admin account if available • Short Terms / personal use footer on site	Security checklist signed off
Day 19	Load & devices	• Test home + subject + PDF on Android (Chrome), iPhone (Safari) • Optional: simple load test on home + one PDF URL	No errors under concurrent refreshes

Day	Focus	Tasks	Done when
Day 20	Content migration	• Upload current batch notes subject-wise (Polity, Current Affairs first) • Pin WhatsApp/Telegram message: one link, how to use tiles	Most daily sends available on site
Day 21	Launch & habit	• Custom domain on Cloudflare (optional) • PWA manifest + Add to Home Screen instructions • Rule: new content = upload first, then one chat message with link • Weekly backup: Postgres export + note R2 bucket name	1000 students have the link; upload routine in place

Daily routine after launch

When	You do	Time
After each class / test	Admin → Upload → subject, day, topic, PDF → Publish → share link in group	5–10 min
Weekly	Check storage meter; unpublish outdated material; backup metadata	15 min
Monthly	Review analytics; top subjects; fix slow pages	20 min

Stack reminder

- **Site:** Astro + Tailwind → Cloudflare Pages (₹0)
- **Catalog + login:** Supabase Postgres + Auth (₹0 at your scale)
- **PDF files:** Cloudflare R2 — 10 GB free, then ~₹50/mo at 50 GB total
- **Students:** no accounts; view/download only